

Abstract Base class & @ abstract method

NOTES

You Can't make objects for this abstract based class

Abstract classes:-

Q. What is an Abstract class?

- A class that cannot be instantiated directly.
- Used to define a common interface for all its subclasses.
- Created using the abc module (ABC class).

Q. Why use it?

- Ensures that subclasses implement specific methods.
- Promotes Code Consistency and reusability.

Q. How to Create one:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def make_sound(self):
```

```
        pass
```

- Animal is abstract
- make_sound() is an abstract method - must be defined in child classes.

Subclass eg:-

```
class Dog(Animal):
```

```
    def make_sound(self):
```

```
        return "Bark"
```

- ✓ Dog implements make_sound(), so it can be instantiated
- ✗ if make_sound() is missing, python will raise an error

◆ Key points:

- Use ABC as base class
- Decorate required methods with @ abstractmethod
- Abstract classes help enforce method contracts across subclasses.

SUMMARY

Getters and setters in python

4-2-2026

NOTES

In python, a getter and setter are methods used to access and update the attributes of a class. These methods provide a way to define controlled access to the attributes of an object, thereby ensuring the integrity of the data.

→ By default attributes in python can be accessed directly.

→ But this may pose problems when attributes need validation or transformation before being assigned or retrieved.

→ Getter:- The getter method is used to retrieve the value of a private attribute

- It allows controlled access to the attribute

→ Setter:- The setter method is used to set or modify the value of a private attribute.

- It allows us to control how the value is updated, enabling validation (or) modification of the data before it's actually assigned.

Python provides several ways to implement getter, setter methods:

1. Using normal function:

In this approach getter & setter methods are explicitly defined to get and set the value of a private variable.

The setter allows setting the value and the getter is used to retrieve it.

class Geek:

```
def __init__(self, age=0):
```

```
    self.age = age
```

#getter method

```
def get_age(self):
```

```
    return self._age
```

Setter method

```
def set_age(self, x):
```

```
    self._age = x
```

```
Raj = Geek()
```

#Setting the age using setter

```
Raj.set_age(21)
```

retrieving age using getter

```
Print(Raj.get_age()) // 21
```

```
Print(Raj._age) // 21
```

._age is a way to create a private attribute

NOTES

Explanation:

- A Greek class is defined with an `_age` attribute
- The `get_age()` method is the getter, which retrieves the value of `_age`
- The `set_age()` method is the setter, which assigns a value to `_age`
- The setter method is used to set the age and the getter method is used to retrieve it.

2. Using `property()` function:

→ In this method, the `property()` function is used to wrap the getter, setter and deleter methods for an attribute, providing a more streamlined approach.

Class Greeks:

```
def __init__(self):
    self._age = 0
```

→ A Greek class is created with an internal `_age` attribute

function to get value of `_age`

```
def get_age(self):
```

```
    print("getter method called")
```

```
    return self._age
```

→ The `get_age()` function retrieves the value

function to set value of `_age`

```
def set_age(self, a):
```

```
    print("setter method called")
```

```
    self._age = a
```

→ # This sets the variable `_age`

function to delete `_age` attribute

```
def del_age(self):
```

```
    del self._age
```

→ # This deletes the attribute

`age = property(get_age, set_age, del_age)` → # The `property()` function binds these methods to the `age` attribute.

```
mark = GreekSC()
```

```
mark.age = 10
```

→ # The `age` attribute is accessed and set using the `property` function.

```
print(mark.age)
```

NOTES

3. Using @property decorators

→ In this approach, the `@property` decorator is used for the getter and the `@<property-name>.setter` decorator is used for the setter.

→ This approach allows a more elegant way to define getter and setter methods.

class Greets:

def __init__(self):

self.age = 0

using property decorator

a getter function

@property

def age(self):

print('getter method called')

return self.age

The Greets use the @property decorator for the getter method, which returns the age.

a setter function

@age.setter

def age(self):

if (a < 18):

raise ValueError("Sorry your age is below eligibility criteria")

print('setter method called')

self.age = a

The @age.setter decorator is used for the setter method, where a validation (age criteria) is added.

The setter is added to assign the value, while the getter retrieves it.

mark = Greets()

mark.age = 19

print(mark.age)

Object introspection

#Feb 5. 2026

NOTES

#Object introspection:-

- It is the ability of a program to examine the type, attributes, methods & properties of objects while it's running (at runtime)
- In python, everything is an object, and introspection allows us to dynamically explore these objects.

Why introspection is useful:-

- Helps understanding object structure & behaviour without reading source code.
- Useful for debugging, logging, dynamic programming, and writing generic code that adapts to objects at runtime.

Key Built-in Functions for introspection:-

① type(Obj)

- Returns the type of the object
- Used to check object type during execution

Python

```
print(type(3)) # <class 'int'>
print(type("Hello")) # <class 'str'>
```

② dir(Obj)

- Lists attributes & methods of an object
- Shows both built-in & custom attributes/methods.

Python

```
Print(dir([1,2,3]))
```

③ str()

- Converts everything into string.

④ id

- This function returns the id of an object

⑤ extra functions etc

Inspect module

#Feb-5-2026

NOTES

Inspect module:-

→ Used to introspect objects and thus provides functions/methods that are used in introspecting

usually two types of methods available:-

Methods to verify the Type of Token

• methods to verify the type of object

method	Description
<code>isclassC obj 1</code>	returns True if obj is class
<code>ismoduleC obj 1</code>	returns True if obj is method
<code>isfunctionC obj, 1</code>	returns True if obj is function
<code>is methodC obj 1</code>	returns True if obj is method

Methods to Retrieve the Source of Token

• Inspect module also provides methods to retrieve the source of an object such as inspecting class hierarchies, member functions, etc

Method	Description
<code>getclasstree(classes)</code>	Returns a nested list of tuples showing the class hierarchy
<code>getmembers(obj)</code>	Returns all the members of an object as (name, value) pairs
<code>signature(callable)</code>	Returns the call signature of a function/method
<code>stack()</code>	Returns the call stack at the point where it's called
<code>getmodule(obj)</code>	Returns the module in which an object was defined
<code>getdoc(obj)</code>	Returns the docstring of the object

Methods to Retrieve the Source of Token:-① `getclasstree(classes)`

• Returns a nested list of tuples showing the class hierarchy

eg: ① In this example we inspect the class hierarchy of C, which inherits from B and B inherits from A.

→ we use `inspect.getmodule(c)` to get method resolution order & pass it to `inspect.classtree(c)` to get a structured inheritance tree.

```
import inspect
class A: pass
class B(A): pass
class C(B): pass

t = inspect.getclasstree(inspect.getmodule(c))
for i in t: print(i)
```

⇒

```
(<class 'object'>, ()),
[(<class '__main__.A'>, (<class 'object'>,)), [(<class '__main__.B'>,
(<class '__main__.A'>,)), [(<class '__main__.C'>, (<class
'__main__.B'>,))]]]
```

② Using `inspect.getmembers()` to fetch members of the math module

NOTES

```
import inspect, math
for n, m in inspect.getmembers(math)[:5]:
    print(n, type(m))
```

Output

```
__doc__ <class 'str'>
__file__ <class 'str'>
__loader__ <class '_frozen_importlib_external.ExtensionFileLoader'>
__name__ <class 'str'>
__package__ <class 'str'>
```

eg 3 Using `inspect.signature()` to get the signature of greet function, showing its parameters & default values.

```
import inspect
def greet(name, age=18):
    return f'Hello {name}, age {age}'
print(inspect.signature(greet)) # name, age = 18
```

eg 4 Using `inspect.stack()` to retrieve the current call stack.

```
import inspect
def test():
    stk = inspect.stack()
    for f in stk[2:]: # Showing top 2 stack frames
        print(f'Function: {f.function}, Line No: {f.lineno}')
test()

Output
Function: test, Line No: 4
Function: <module>, Line No: 8
```

eg 5 `inspect.getmodule` to retrieve the module

```
import inspect
import math

m = inspect.getmodule(math.sin)
print(m.__name__)
```

Output

math

eg 6 `inspect.getdoc()`
↳ gets the docstring of the function

#Generators in python

6-Feb-2026

NOTES

Generator in python

• A generator function is a special type of function that returns an iterator object

→ Instead of using **return** to send back a single value, generator functions use **yield** to produce a series of results over time

→ The function pauses its execution after **yield**, maintaining its state between iterations.

eg

```
def fun(max):
    cnt = 1
    while cnt <= max:
        yield cnt
        cnt += 1
    cnt = fun(5)
    for n in cnt: print(n)
```

⇒ op

1
2
3
4
5

Exp: This generator function **fun** yields numbers from 1 to max. Each call to **next()** on the generator obj resumes execution right after the **yield** statement, where it left off.

Why do we need generators?

• **Memory efficient**: Handle large or ∞ data without loading everything into memory.

• **No list overhead**: Yield items one by one, avoiding full list creation.

• **Lazy evaluation**: Compute values only when needed, improving performance.

• **Support of sequences**: ideal for generating unbanded data like Fibonacci series.

• **pipeline processing**: Chain generators to process data in stages efficiently.

Creating Generators

```
def gen(parameters):
    #code
    yield expression
    #code
```

eg:

```
def fun():
```

```
    yield 1
```

```
    yield 2
```

```
    yield 3
```

#driver code to check above gen function

```
for val in fun():
    print(val)
```

NOTES

Yield vs Return:-

Yield: to provide sequence of values over time

Yield executed
↓
function Paused → returns Current Value
↓
function retains

→ This allows the function to continue from

Same point when called again. making it

ideal for generating large or complex sequences efficiently.

Return :-

to exit a function & return a final value

Return Executed
↓
func terminated → output: result

→ Suitable for cases where a single result is needed from a function.

Generator Expression

→ This is a concise way to create generators.

→ Similar to list comprehensions but use parentheses and are more memory efficient.

Syntax:-

(expression for item in iterable)

eg:

```
Sq = (x*x for x in range(1,6))

for i in Sq:
    print(i)
```

Applications of Generators in python.

→ Suppose we need to create a stream of Fib numbers. using a generator makes this easy, you just call next() to get the next number without worrying about the stream ending.

→ Generators are especially useful for processing large data files like logs, because they handle data in small parts, saving memory and no need to load the entire file at once.

→ While iterators can do similar tasks, generators are quicker way to write since we don't need to define __next__ and __iter__ methods manually.